

Escuela de
Ingeniería y Arquitectura
Universidad Zaragoza

Universidad de Zaragoza

Facultad de Ingeniería

Artificial Intelligence - Lab Session 3: Neural Networks and CNNs

Jorge Ruiz González (826685)
Hugo López Sola (875455)

October 14th, 2026

Contents

1 Aim and evaluation **2**

2 Task 1: Two-layer neural network **3**

3 Task 2: Simple CNN **4**

4 Task 3: Transfer learning and fine-tuning **5**

1 Aim and evaluation

In Task 1, we first train the two-layer network on the synthetic toy data, then on CIFAR-10. For CIFAR-10 we use 49,000 training images, 1,000 validation images and 10,000 test images. In Tasks 2 and 3, we train CNNs on five food classes: cheesecake, chicken quesadilla, chicken wings, churros and donuts. There are 1,000 images per class, split into 4,000 training and 1,000 validation images with seed 123.

We use validation accuracy to compare the models. The food dataset has no separate test set. Its results cannot be directly compared with CIFAR-10, since the images and classes are different.

2 Task 1: Two-layer neural network

The network applies a fully connected layer, ReLU and a second fully connected layer. The last layer produces class scores; softmax cross-entropy measures the classification error. L2 regularization penalizes large weights, and stochastic gradient descent updates the parameters using minibatches.

On the five-sample, three-class toy problem, training reaches a loss of 0.0367, below the requested 0.2. The toy initialization uses a standard deviation of 0.1; CIFAR-10 retains the starter's default initialization. CIFAR images are flattened into 3,072 features and centred using only the training mean.

The initial CIFAR model uses 50 hidden units and 1,000 iterations, obtaining 28.60% validation accuracy. We then run 20 random trials of 2,000 iterations, using learning rates $\{10^{-4}, 5 \times 10^{-4}, 10^{-3}\}$, hidden sizes $\{50, 100, 150\}$ and regularization strengths $\{0.25, 0.5, 0.75\}$. We leave out 10^{-2} because it caused numerical overflow in the earlier trials.

Table 1: Selected two-layer network on CIFAR-10.

Configuration	Training	Validation	Test
100 hidden units, $lr = 10^{-3}$, $reg = 0.25$	54.52%	52.50%	49.96%

We evaluate the selected model on the test set only after the search. To reduce overfitting, we would try more training data or stronger L2 regularization. Adding hidden units can make overfitting worse. A smaller learning rate changes the training updates but does not directly solve this problem.

3 Task 2: Simple CNN

The baseline resizes food images to 150×150, rescales their intensities and uses three convolutional/pooling blocks with 16, 32 and 64 filters, followed by a 128-unit dense layer and five output scores. We compare it with dropout, then add horizontal flips, small rotations, zoom and contrast changes. The larger model uses 32/64/128 filters and 256 dense units; all kernels remain 3×3.

Table 2: Final-epoch results saved in the main Task 2 notebook.

Model	Epochs	Training	Validation
Baseline	10	99.10%	41.30%
Dropout	10	72.35%	48.80%
Dropout + augmentation	20	52.93%	53.60%
Larger CNN + augmentation	20	56.15%	54.40%

Without dropout, training accuracy is almost 100%, but validation accuracy is only 41.30%. This shows clear overfitting. Dropout reduces the gap and raises validation accuracy to 48.80%.

With augmentation, validation accuracy is 47.60% at epoch 10 and 53.60% at epoch 20. We therefore cannot attribute the whole improvement to augmentation: this model also trains for longer. Its training and validation accuracies are similar, but training uses dropout and augmented images while validation does not. This alone is not enough to conclude that the model underfits.

The separate longer-run notebook reaches 57.40% final validation accuracy with the larger CNN after 40 epochs (61.30% at its best epoch). Since this is a separate run, the difference is not a controlled comparison of epoch counts.

4 Task 3: Transfer learning and fine-tuning

We load MobileNetV2 with ImageNet weights and remove its original classifier. Images are resized to 224×224 and receive the model's matching preprocessing. Global average pooling, dropout and a five-output dense layer form the new classifier. The output is logits, so sparse categorical cross-entropy uses `from_logits=True`.

First, the backbone is frozen and the new classifier is trained with Adam at 10^{-3}. We then unfreeze the backbone and recompile at 10^{-4}. Two variations restart from the same saved frozen-head weights: full fine-tuning at 10^{-5}, and fine-tuning only the last 20 backbone layers at 10^{-5}, with BatchNormalization layers frozen. Backbone BatchNormalization statistics remain in inference mode throughout, as in the starter.

We run five epochs per stage on CPU, with seed 123 and batch size 4. Both variations use new optimizers and callbacks. We save the weights with the lowest validation loss and set early-stopping patience to five epochs. Table 3 shows the final epoch and the selected checkpoint separately, since they are not always the same.

Table 3: Task 3 results. Selected checkpoints minimize validation loss.

Stage, learning rate	Epochs	Final train	Final val.	Selected val.
Frozen backbone, 10^{-3}	5	89.92%	90.90%	–
Full fine-tuning, 10^{-4}	5	90.50%	71.70%	–
Full fine-tuning, 10^{-5}	5	82.13%	79.10%	88.30%
Last 20 layers, 10^{-5}	5	95.95%	93.10%	93.10%

The frozen backbone already gives 90.90% validation accuracy. Unfreezing all layers at 10^{-4} makes the result worse: it falls to 71.70%, even though training accuracy reaches 90.50%. Lowering the learning rate to 10^{-5} gives 88.30% at the selected checkpoint, but only 79.10% at the final epoch.

Training only the last layers works best here, reaching 93.10%. These results do not support unfreezing every layer by default. We would keep the partial fine-tuning model for this dataset. The comparison uses one split and five epochs per stage, so it does not show which strategy would always be best.

The ImageNet weights are necessary for this transfer-learning experiment. Loading the architecture with random weights would mean training from scratch. Compared with the simple CNN's 54.40%, the pretrained model gives much higher validation accuracy, although the image resolution and training budgets differ. It also requires the correct preprocessing and a suitable fine-tuning learning rate.